Developer Reflection Journal

Justin Ramas | John Anthony Romeo | Joel Franco Navales

***N**early **A**ll **P**rogrammers **S**leeping*

Our team experienced incredible growth and technical success while building this application with Flutter and Supabase, but we also had to navigate some very steep engineering challenges along the way. What worked exceptionally well for our group was establishing a clean architecture pattern right from the start, which kept our repository interfaces completely separate from our concrete Supabase implementation files. This disciplined approach paid off immediately when our comment_likes database table needed to be restructured, because only the implementation files had to be changed while the rest of our application code remained completely untouched. Additionally, utilizing Riverpod with code generation made our application state highly predictable and saved us a tremendous amount of development time, while keeping our individual features organized in separate folders ensured that we never accidentally broke other parts of the app while working concurrently. The post feed feature also came together much faster than we originally expected, allowing us to easily connect Supabase Storage with our image uploads and cleanly render all the user posts in reverse chronological order. We also found that using optimistic UI updates for likes and comments made the entire application interface feel instantaneous and highly responsive, which successfully eliminated the need for annoying loading spinners during frequent user interactions. Furthermore, leveraging artificial intelligence tools throughout the development cycle served as a massive advantage, helping our team move quickly through repetitive boilerplate code, debug confusing PostgREST errors regarding missing foreign key hints, and safely verify our security rules before deployment. Despite these major victories, implementing certain advanced Supabase features proved to be highly challenging, with our biggest headache being the complex follow and unfollow system. This feature was particularly difficult because we constantly

tripped over the self-referencing foreign key relationships where both the follower identification and the following identification pointed directly back to the exact same users table. Writing the Row-Level Security policies for this configuration took us several frustrating attempts to get right, and we frequently found ourselves completely locked out of our own data tables during testing. We also encountered a brutal technical hurdle trying to get our real-time database updates and local optimistic state updates to coexist without duplicating the like counts. When a user tapped the like button, our system updated the user interface immediately, but Supabase would simultaneously broadcast a real-time insert event for that same interaction, causing a race condition under realistic network lag that tried to increment the counter twice. To resolve this, we had to engineer a custom idempotency guard inside our event handling logic to detect and skip real-time events that had already been applied locally. We faced additional hurdles handling user authentication persistence to stop the app from logging users out whenever it restarted, which required us to bind our authentication state stream directly to our navigation system, and we struggled with building an atomic workflow to ensure that profile picture storage uploads and database updates succeeded together. Reflecting on these roadblocks, if we were to do this project over again, we would completely change our initial approach by dedicating our entire first day to collaboratively designing the database schema and security policies on paper before writing a single line of Flutter code, since most of our major development blockers came from schema changes we were forced to bolt on later in the cycle. We would also establish a unified custom exception hierarchy across the entire app instead of leaving raw exceptions in the likes module, utilize Supabase's built-in exact count feature to stop downloading entire data rows just to count them, and write thorough inline documentation to explain our complex real-time race condition code. This successful product delivery was made possible by distinct individual contributions, as Justin and Ramas completely owned the core authentication flows, user profile features, navigation persistence, and

the final integration pass that pulled our modules together into a working build. Joel took full responsibility for building the post feed and setting up the backend database relationships for the follow system, while John established our overall clean architecture boundaries and engineered the vital idempotency logic to solve our real-time network bugs. Finally, Romeo took end-to-end ownership of the interactive platform elements by designing and building all the frontend interfaces and backend database structures for both the likes and comments features. By combining our specialized strengths, we successfully overcame our data engineering obstacles and delivered a unified, high-performing application.

Individual Reflections

Justin's Reflection:

- I took ownership of the login and user profile features for our project. Using Riverpod with code generation made state management very predictable and saved me a lot of time compared to doing it manually. Keeping my work in separate folders also helped because I never accidentally broke other parts of the app. My biggest regret was how we handled the database structure. We wasted a lot of time fixing errors and rebuilding tables mid-project, so I wish we had spent our entire first day designing the database schema together. I also faced a few tough challenges. I had to fix an issue where users were logged out when restarting the app by linking the authentication stream to our navigation logic. On top of that, handling profile picture uploads was tricky because I had to make sure both the cloud storage upload and the database update succeeded together.

Joel's Reflection:

- What worked well for our team was the post feed feature — getting Supabase Storage to play nicely with our image uploads and rendering everything in reverse-chronological order came together faster than expected, and the realtime updates on likes were a satisfying win. The hardest Supabase feature to implement was definitely the follow and unfollow system, mainly because we kept tripping over the foreign key relationships between follower_id and following_id both pointing back to the same users table, and the RLS policies for it took a few tries to get right without locking ourselves out of our own data. If we did this again, we'd spend more time sketching out the database schema and RLS policies on paper before writing a single line of Flutter code, because most of our blockers came from schema changes we had to bolt on later. Contribution-wise, I took responsibility for the post feed and the follow/unfollow system, Romeo handled the likes and comments features end-to-end, while Ramas owned the auth flow (login and sign-up), the user profile screen, and the final integration pass that pulled everything together into a working build. Using AI was a real advantage for this project — it helped us move faster on boilerplate, debug confusing PostgREST errors like missing foreign key hints, and sanity-check our RLS policies before we deployed them, which honestly saved us hours of trial-and-error in the Supabase dashboard.

John's Reflection:

- The clean architecture (repository interfaces separate from Supabase implementations) paid off immediately when the comment_likes join needed restructuring, only the impl file changed. Optimistic updates on likes and comment likes made the UI feel instant, and seeding the like count from feed data meant no loading spinners on the most-used interaction. Getting realtime and optimistic updates to coexist without doubling the like count. When a user taps like, the notifier updates state

immediately and Supabase fires a realtime INSERT both trying to increment the same counter. The fix was an idempotency guard in _handleLikeChange() that skips realtime events already applied optimistically. It sounds simple but the bug only showed up under realistic network lag, not in local testing. Add a proper exception hierarchy to the likes side (the comments side has one, likes just re-throws raw Dart exceptions). Use Supabase's count: exact instead of fetching full rows to count them. And document the realtime/optimistic race condition inline while it's fresh — that logic is the hardest part for anyone reading the code cold.